

Estudio IA · Salarios de convenio

Telmo

12 agosto 2026

CONTENTS

Guion del estudio — “Los asistentes de IA se equivocan con tu convenio”	
1. Diseño de la matriz	
2. Muestra de convenios (recomendada, ajustable)	
3. Prompt fijo	
4. Rúbrica de evaluación (por respuesta)	
5. Ground-truth automatizable (lo que sí puedo construir hoy, sin API externa)	
6. Captura de respuestas (requiere API — §7)	
7. Prerrequisitos y bloqueos	
8. Validez y honestidad metodológica (para que la pieza aguante escrutinio)	
9. Cadencia y entregables	
10. Próximos pasos concretos (en orden)	

Guion del estudio — “Los asistentes de IA se equivocan con tu convenio”

Estado: **PREPARADO, no ejecutado**. Bloqueante: claves de API (§7). Deriva de la pieza de sala de prensa §5. Complementa `press/evidencia-ia-2026-08/` (caso n=1). Fecha guion: 12-ago-2026.

El objetivo de esta extensión es convertir la anécdota (n=1, Baleares) en **dato de prensa reproducible y recurrente**: medir con qué frecuencia y de qué forma los asistentes de IA se equivocan al responder salarios de convenio, evaluados contra tablas verificadas contra boletín.

Titular publicable objetivo: “El X % de las respuestas de IA sobre salarios de convenio contienen al menos un error” + ranking por asistente + tipología de errores.

1. Diseño de la matriz

4 asistentes × 10 convenios × 3 repeticiones (días distintos) = 120 respuestas por ola.

Eje	Valores	Nota
Asistente	ChatGPT · Claude · Gemini · Perplexity	Con búsqueda web activada (es el modo en que un trabajador real pregunta). Registrar también variante sin web si el API lo permite.
Convenio	10 fichas del catálogo (§2)	Todas con tabla verificada en <code>data/convenios/*.json</code> .
Repetición	3 días distintos	Captura la varianza no-determinista del modelo + rotación de fuentes indexadas.

Cada respuesta se guarda cruda (texto completo + fuentes citadas + timestamp + prompt exacto) antes de puntuar. Un dato sin su respuesta cruda archivada no cuenta (regla: no reconstruir de memoria).

2. Muestra de convenios (recomendada, ajustable)

Criterio de selección: **maximizar la superficie de error estructural** que la pieza denuncia (matriz nivel×categoría, salario diario, paradoja SMI, obsolescencia por ultraactividad), más un control “fácil” para tener línea base. No es una muestra aleatoria: es un *stress test* dirigido.

#	Ficha	JSON	Trampa que expone
1	Hostelería Baleares	hosteleria_baleares.json	Caso insignia. Matriz Nivel I-V × Categoría A/B/C → colapso a “un rango”. Año fiscal abr-mar.
2	Hostelería Madrid	hosteleria_mad.json	Referencia canónica, altísima demanda → más fuentes que resumen mal.
3	Metal Sevilla	metal_sev.json	Salario diario A/B/C + anual estimada → error de conversión diario↔mensual↔anual.
4	Limpieza Tenerife	limpieza_tf.json	Paradoja SMI (anual bajo 17.094 €) → IA suele “corregir al alza” al SMI y falla.
5	Limpieza Pontevedra	limpieza_pontevedra.json	16 pagas, sin paradoja → contraste con #4: ¿la IA distingue nº de pagas?
6	Metal Álava	metal_ala.json	Foral (BOTH, no BOE), ultraactividad, tablas 2026 +0,5% → riesgo de citar tabla vieja.
7	Hostelería Bizkaia	hosteleria_bizkaia.json	Foral (BOB). ¿Atribuye bien el boletín provincial vs “BOE”?
8	Construcción Madrid	construccion_mad.json	Otro sector, otra estructura de grupos → generaliza el hallazgo fuera de hostelería.
9	Contact Center estatal	contact_center_estatal.json	Marco estatal de tabla simple → control : aquí la IA debería acertar.
10	Limpieza Madrid	limpieza_mad.json	Alta demanda + estructura de categorías → segundo sector de gran volumen.

De las 51 fichas del censo, 13 están en **ultraactividad** — son las de mayor riesgo de cita obsoleta y el mejor caldo para el hallazgo “cita de 2003”. Si se amplía la muestra, priorizar esas 13 sobre las recién publicadas.

Decisión pendiente para Telmo: ¿fijamos estos 10 o quieres sustituir alguno? El único obligatorio es Baleares (#1), porque es el caso ya documentado que ancla la pieza.

3. Prompt fijo

Un único prompt, idéntico salvo las variables `[puesto]`, `[sector]`, `[provincia]`, para que las respuestas sean comparables. Se ejecuta tal cual lo escribiría un trabajador (sin pedir fuentes ni “contra el boletín” — eso sesgaría el test a favor).

¿Cuánto cobra un/a `[puesto]` según el convenio de `[sector]` de `[provincia]` en 2026?

- El `[puesto]` se elige por convenio para forzar la casilla exacta de la matriz (p. ej. Baleares: “camarero/a de hotel de 1 estrella” → obliga a Nivel IV, categoría C = casilla única, no rango). La tabla de puestos-diana por convenio se define en `objetivos.json` (§5) para que la casilla esperada sea inequívoca.
- Se registra una **segunda variante** de control por convenio con el puesto genérico (“camarero/a”) para medir si la ambigüedad del prompt explica parte del error (atribución justa al asistente).

4. Rúbrica de evaluación (por respuesta)

Cada respuesta se puntúa en 6 dimensiones binarias/ordinales. La puntuación se deriva **automáticamente** comparando contra el ground-truth del JSON (§5); las dimensiones no automatizables se marcan para revisión humana ligera.

#	Dimensión	Valores	Auto?	Fuente de verdad
D1	¿Convenio correcto?	sí / no / ambiguo	semi	nombre + código convenio del JSON
D2	¿Tabla vigente (tramo correcto)?	sí / obsoleta / no verificable	semi	<code>vigencia</code> + año/tramo del JSON
D3	¿Cifra exacta?	exacta / dentro ± 2 % / fuera / rango-en-vez-de-cifra	sí	<code>grupos[].salario[año].mes</code>
D4	¿Estructura nivel/categoría respetada?	sí / colapsada a rango / inventada	sí	nº de grupos/subtablas del JSON
D5	¿Pluses correctos?	correctos / inventa plus inexistente / omite / imprecisos	semi	<code>complementos</code> del JSON
D6	Trazabilidad de fuentes	cita boletín / cita agregador / cita calculadora / sin fuente	manu- al	texto de la respuesta

Métrica principal publicable = % de respuestas con ≥ 1 error (D1..D5 con valor distinto de “correcto”). Métricas secundarias: exactitud de cifra (D3), tasa de “rango en vez de cifra” (D4), tasa de plus inventado (D5), ranking por asistente, evolución trimestral.

D3 — comparación de cifra (el corazón automatizable):

- `exacta` : coincide con `mes` del grupo-diana (tolerancia de redondeo $\pm 0,50$ €).
- `dentro  $\pm 2$  %` : cerca pero no exacta (típico de fuente obsoleta de un tramo anterior).
- `rango-en-vez-de-cifra` : la respuesta da un intervalo A–C cuando el puesto-diana tiene casilla única (**el error del caso C de Baleares**). Se detecta si la respuesta contiene dos números que coinciden con el mínimo y máximo de la subtabla en vez del valor de la casilla.
- `fuera` : no coincide con ninguna casilla vigente del convenio.

5. Ground-truth automatizable (lo que sí puedo construir hoy, sin API externa)

Todo el material de referencia **ya existe** en `data/convenios/*.json`. La parte automatizable del estudio no depende de las claves de API y se puede dejar lista y testeada ya:

1. `objetivos.json` — por cada uno de los 10 convenios: `{ slug, sector, provincia, puesto_diana, grupo_id_esperado, año/tramo, cifra_esperada, subtabla, min_subtabla, max_subtabla, pagas, complementos_reales[], plus_trampa[] }`. Se genera leyendo los JSON; ningún número se teclea a mano.
2. `extraer-ground-truth.js` — script determinista que puebla `objetivos.json` desde los JSON del catálogo y falla ruidosamente si un grupo-diana no existe (evita objetivos fantasma).
3. `evaluar.js` — recibe las respuestas crudas (`respuestas/*.json`) y aplica D3/D4 de forma automática contra `objetivos.json`; emite `resultados.csv` + agregados. D1/D2/D5/D6 quedan como columnas para completar (semi/manual).

Recomendación: construir 1–3 **primero** (cero dependencias externas, valida la muestra y las casillas-diana), y dejar la captura vía API (§6) para cuando haya claves. Así el guion queda “armado y probado en seco” antes de gastar un euro de API.

6. Captura de respuestas (requiere API — §7)

Un `capturar.mjs` por asistente que, para cada (convenio × repetición), lanza el prompt fijo con búsqueda web y guarda `{ asistente, convenio, repeticion, timestamp, prompt, respuesta_texto, fuentes_citadas[] }` en `respuestas/`. Consideraciones:

- **Claude:** API Anthropic (ver skill `claude-api` para modelo/params vigentes). Web search vía tool.
- **ChatGPT / Gemini / Perplexity:** cada uno con su propia clave y su modo de búsqueda web.
- No hay determinismo: por eso 3 repeticiones en días distintos, no 3 seguidas.
- Coste estimado por ola: 120 respuestas × 4 asistentes ya contadas = **480 llamadas** (4×10×3× variante-control ≈ 240 principales + 240 control). Cerrar presupuesto antes de ejecutar.
- Archivar SIEMPRE la respuesta cruda + fuentes antes de puntuar. Sin cruda archivada, la fila no existe.

7. Prerrequisitos y bloqueos

Prerrequisito	Estado hoy	Acción
Claves de API (OpenAI, Google/Gemini, Perplexity, Anthropic)	Ausentes (solo Claude Code en env)	Telmo aporta claves o decide asistentes a incluir.
<code>objetivos.json</code> + extractor	No existe	Se puede construir ya (§5), sin depender de nada.
Muestra de 10 fijada	Propuesta (§2), pendiente OK	Confirmar o ajustar.
Presupuesto de API por ola	Sin cerrar	Estimar antes de lanzar.
Encaje con ventana pre-reapelación AdSense	El estudio no publica hasta tener n	Ejecutar captura no toca el sitio; publicar la pieza sí → decisión aparte.

8. Validez y honestidad metodológica (para que la pieza aguante escrutinio)

- **n y potencia:** 10 convenios no es representativo del universo de convenios de España; es una muestra dirigida a estructuras difíciles. La pieza debe decirlo: “seleccionamos 10 convenios que concentran las trampas estructurales típicas”, no “los convenios españoles”.
- **Atribución justa:** la variante-control de prompt genérico (§3) separa “la IA falla” de “el prompt era ambiguo”. Publicar ambas cifras.
- **Deriva temporal:** las fuentes indexadas cambian; una ola es una foto. Por eso la cadencia trimestral y la serie temporal (“¿mejoran los asistentes?”).
- **Reproducibilidad:** publicar prompts exactos, fechas, y el CSV de resultados como anexo descargable (§4). El cotejo se hace contra boletín (edicto + fecha), no contra nuestra propia ficha.
- **No inventar:** si una respuesta es ambigua de puntuar, se marca “no verificable”, nunca se fuerza a una casilla. Regla dura del proyecto.

9. Cadencia y entregables

- **Ola 0 (piloto):** los 10 convenios × 4 asistentes × 1 repetición = 40 respuestas, para validar rúbrica y tubería antes de gastar en 3 repeticiones.
 - **Ola 1 (completa):** matriz 120, publicable.
 - **Trimestral:** repetir → serie temporal. Picos de interés: enero (SMI/revisiones) y cualquier hito de la directiva de transparencia.
 - **Entregables por ola:** `resultados.csv` (anexo descargable), gráfico ranking por asistente, tipología de errores, y actualización de la serie temporal en la pieza.
-

10. Próximos pasos concretos (en orden)

1. Telmo fija/ajusta la muestra de 10 (§2) — solo Baleares es obligatorio.
2. Construir `extraer-ground-truth.js` → `objetivos.json` (sin API, hoy mismo).
3. Definir puestos-diana por convenio (casilla inequívoca) dentro del extractor.
4. Construir `evaluar.js` (D3/D4 automáticos) + tests contra el caso Baleares ya documentado.
5. Cuando haya claves: `capturar.mjs`, cerrar presupuesto, lanzar Ola 0.
6. Revisar Ola 0 a mano, ajustar rúbrica, lanzar Ola 1, redactar resultados.